Progettazione e implementazione di un Hybrid Multilayer Firewall

Costanza Mancini (637274)

10 novembre 2026

Indice

1 Obiettivo del progetto **2**

2 Architettura e flusso di elaborazione **2**

3 Scelte progettuali **2**

4 Analisi del traffico **3**
4.1 Analisi di pacchetto . . . 3
4.1.1 Controlli generali . . . 3
4.1.2 Controlli di rete . . . 4
4.1.3 Controlli di trasporto . . . 4
4.2 Analisi di flusso . . . 4

5 Policy Engine **5**
5.1 Ordine di valutazione e politica di default . . . 5
5.2 Regole su pacchetto, flusso e finestra temporale . . . 6
5.2.1 Regole per singolo pacchetto . . . 6
5.2.2 Regole per flusso . . . 7
5.2.3 Regole per finestre temporali . . . 7
5.3 Esito della valutazione . . . 8

6 Testing e risultati **8**
6.1 Suite di test . . . 8
6.2 Risultati . . . 10

7 Conclusioni **10**

1 Obiettivo del progetto

Il progetto prevede la progettazione e l'implementazione di un *Hybrid Multilayer Firewall*, combinando l'analisi del traffico di rete con una serie di decisioni di filtraggio personalizzate.
Lo scopo del progetto rimane puramente accademico ma l'intenzione è quella di sfruttare la raccolta dei dati del traffico e la loro analisi a scopi di sicurezza, per individuare anomalie o attacchi specifici.
La decisione principale è stata quella di affiancare due livelli di analisi, quella dei singoli pacchetti e quella di flusso, in modo da tenere conto del comportamento complessivo del traffico e valutare un insieme più ampio di situazioni anomale. L'analisi viene svolta in locale e offline su traffico precedentemente acquisito (tramite strumenti come Wireshark) e salvato all'interno di file PCAP.
Il componente fondamentale del firewall è rappresentato dal *Policy Engine* che, a partire dalle informazioni raccolte, applica un insieme di regole stabilite per assegnare ad ogni pacchetto un esito di tipo ALLOW o DROP. Il comportamento del sistema è stato valutato principalmente tramite una suite di 17 scenari semplificati, di cui sono riportati i risultati, ma anche da una serie di catture di traffico di una rete domestica.

2 Architettura e flusso di elaborazione

L'architettura del firewall è stata progettata su più componenti ognuno con una responsabilità ben precisa nel flusso di elaborazione delle informazioni catturate.
L'Orchestrator è l'elemento che si occupa del coordinamento delle operazioni, stabilendo il corretto ordine delle fasi e mantenendo il collegamento tra il pacchetto catturato, il suo eventuale flusso di appartenenza e la decisione finale.
I pacchetti acquisiti vengono dati in pasto ad una funzione che utilizza PcapReader, una sorta di iteratore sul file di cattura, che li estrae in modalità streaming, per elaborarli uno alla volta e prenderne i campi d'interesse, come indirizzi MAC e IP, protocollo di trasporto, porte, flag TCP, timestamp e lunghezze disponibili. Le informazioni estratte vengono raccolte in un oggetto Packet, che mantiene l'indice utile a identificarne la posizione temporale nel PCAP. Il Packet Analyzer riceve tutti i pacchetti nell'ordine originale di cattura e produce per ciascuno un risultato dettagliato con i campi osservati e l'esito dei controlli effettuati.
L'orchestratore si occupa di costruire una vista dei pacchetti ordinata per timestamp, utilizzando l'ordine di cattura a parità di timestamp. Il Flow Analyzer riceve la vista e per ogni pacchetto, quando possibile, individua il flusso di appartenenza. Si riesce a costruire un collegamento per risalire dal singolo pacchetto allo stato e statistiche di flusso elaborate dall'analyzer, raccolti in un oggetto Flow.
All'interno di un CaptureAnalysis i pacchetti vengono raccolti in base all'ordine di cattura, con i risultati di analisi singole e dei flussi finalizzati. Il Policy Engine applica la configurazione di filtraggio stabilita e produce per ciascun pacchetto un giudizio affermativo o negativo, insieme alla relativa motivazione.

3 Scelte progettuali

Per l'implementazione del sistema, al fine di prediligere semplicità nella prototipazione, leggibilità e la modularità architetturale, è stato scelto Python come linguaggio. Il suo utilizzo ha permesso di definire in modo chiaro le strutture dati scambiate tra i componenti e utilizzare di conseguenza la libreria Scapy per la lettura e la decodifica dei pacchetti. Scapy mette a disposizione un'interfaccia flessibile e intuitiva per accedere direttamente a diversi e numerosi livelli protocollari e ai relativi campi attraverso poche istruzioni Python. L'intenzione è stata quella di evitare l'implementazione manuale di meccanismi di decodifica affidando completamente la decodifica a Scapy e permettendo di concentrare lo sviluppo sui controlli e sulle regole. Come spiegato nel paragrafo precedente, viene messa a disposizione la funzione PcapReader per la lettura progressiva dei PCAP: il suo utilizzo consente di contenere l'occupazione della memoria durante la lettura dei file, soprattutto in visione di catture realistiche voluminose, senza richiederne il caricamento integrale. L'utilizzo di Scapy porta con sè degli aspetti negativi in termini di throughtput e scalabilità. Rispetto a librerie a basso livello in C come libpcap infatti risulta molto meno efficiente e presenta un overhead significativo in termini di spazio e tempo. Nello sviluppo sono stati effettivamente riscontrati i limiti di questa

libreria: la gestione di pacchetti troncati, incoerenti, non supportati o anche malformati è stata gestita esplicitamente, attraverso campi adibiti per distinguere la dimensione effettivamente acquisita da quella originale oppure con controlli sulle lunghezze di header, payload, campi TCP/UDP e coerenza di protocolli. Nel caso di protocolli di trasporto non riconosciuti o non supportati è stato previsto un valore OTHER. La semplice decodifica di Scapy considera queste anomalie motivi di scarto.
Si è preferito sviluppare il progetto come un sistema di analisi offline e locale soprattutto allo scopo di contenere la complessità implementativa.
L'elaborazione offline è utile per concentrarsi su catture già concluse e in particolare tutte quelle caratteristiche aggregate ricavabili dai flussi, senza il vincolo dell'effettiva velocità di trasmissione dei pacchetti. In questo modo si ha la possibilità di introdurre regole di filtraggio anche molto restrittive senza andare ad introdurre ritardi nella trasmissione o comprometterne in generale il funzionamento, magari a causa di decisioni errate.
Uno dei principi chiave seguito nella progettazione è stato quello della riproducibilità degli esperimenti; l'offline riesce a dare la possibilità di valutare le stesse catture con configurazioni di filtraggio differenti, come regole o soglie, o profili di scenari diversi, confrontando i risultati ottenuti.
L'esecuzione in locale ha invece permesso di escludere tutta quella parte di sviluppo di aspetti riguardanti la riservatezza dei dati o la gestione di servizi remoti, rendendo l'ambiente maggiormente controllabile. L'utilizzo dei file PCAP per conservare tracciati di traffico reali o semplificati è stata una conseguenza implementativa di questi aspetti. L'esigenza di disporre pacchetti precedentemente acquisiti con una rappresentazione persistente è stata soddisfatta tramite questo formato standard, che permette di memorizzare informazioni di rete conservandone la struttura originale per poi valutarne aspetti quantitativi e qualitativi. Queste scelte introducono ovviamente alcune limitazioni, prima tra tutte l'impossibilità di non intervenire direttamente sul traffico in corso, non bloccando i pacchetti attraverso le decisioni e di conseguenza non riconoscendo subito comportamenti malevoli. Si tiene anche conto del fatto che il risultato prodotto dalle regole dipende dalla qualità della cattura e dalla porzione osservata della comunicazione, non considerando tutti i pacchetti effettivamente trasmessi. Inoltre non si considerano in alcun modo informazioni specifiche riguardo stato corrente della rete e dei suoi host.
Per questi motivi, il sistema rimane semplicemente di carattere valutativo senza determinare un'effettiva modifica del traffico di rete, che porterebbe ovviamente ad avere una protezione attiva.

4 Analisi del traffico

L'analisi del traffico di rete è stata fatta su due livelli in modo da distinguere le informazioni ricavabili dall'elaborazione del singolo pacchetto da quelle che richiedono l'osservazione nel tempo di aggregazioni di pacchetti. I risultati ottenuti vengono mantenuti distinti grazie alle strutture dati adibite e successivamente dati in input per le decisioni di filtraggio.

4.1 Analisi di pacchetto

L'analisi di pacchetto ha l'obiettivo di verificare la coerenza dei campi disponibili e di descrivere caratteristiche osservabili. Il PacketAnalyzer riceve un oggetto Packet ed effettua una serie di controlli. Ogni analisi di pacchetto restituisce un oggetto PacketAnalysis contenente informazioni generali con indirizzi e protocolli, esiti dei controlli e caratteristiche significative rilevate. Gli esiti sono:

- VALID: i dati rispettano i vincoli;
- INVALID: i dati presentano una violazione sui controlli;
- INCOMPLETE: i dati disponibili non consentono di completare la verifica.

Casi in cui i controlli non sono pertinenti restituiscono esito NOT APPLICABLE mentre per controlli non eseguiti si dà esito NOT CHECKED.

4.1.1 Controlli generali

I controlli generali riguardano timestamp, lunghezza di pacchetto e payload e coerenza tra le informazioni fornite dai livelli di protocollo. Per il timestamp, l'assenza risulta in un controllo incompleto mentre valori non numerici, negativi o non finiti sono invalidi. Lunghezze di pacchetto e

di payload vengono esaminate in base ai byte effettivamente acquisiti, la dimensione originale riportata nei metadati della cattura e le lunghezze dichiarate nelle intestazioni: questa differenziazione viene applicata al fine di verificare la coerenza strutturale del pacchetto e gestirli correttamente, distinguendo pacchetti troncati con dati non disponibili da pacchetti con campi che dichiarano dimensioni incoerenti rispetto a quelle previste dal protocollo.

4.1.2 Controlli di rete

A livello di rete si controllano i corretti formati degli indirizzi e la corrispondenza rispetto alla versione IP indicata e la lunghezza dell'intestazione. I controlli per IPv4 e IPv6 sono distinti dato che per il primo è prevista un'intestazione di lunghezza variabile mentre per il secondo si ha un'intestazione base di 40 byte eventualmente seguita da intestazioni di estensione. Il controllo per IPv6 è mirato alla coerenza della catena di intestazioni: il valore finale contenuto in Next Header viene confrontato con il protocollo identificato per rilevare incongruenze sulle informazioni delle intestazioni e sulla struttura del pacchetto, distinguendo casi con protocolli diversi da casi con informazioni insufficienti.
Un altro aspetto gestito è quello della frammentazione, limitandosi ad individuarla attraverso i campi dell'offset e le indicazioni di eventuali frammenti, in modo da distinguere pacchetti non frammentati, da pacchetti con informazioni di frammentazione valide a quelli per cui non se ne può verificare il contenuto. Non è stato introdotto nessun meccanismo di riassemblaggio IP che richiederebbe di raccogliere tutti i frammenti appartenenti ad uno stesso datagramma, ordinarli e ricostruire il contenuto originale.

4.1.3 Controlli di trasporto

A livello di trasporto si scindono le caratterizzazioni in base al protocollo identificato, effettuando sia controlli di validità, su completezza e coerenza dei campi, che classificazioni di proprietà utili al filtraggio.
Su TCP e UDP è stata prevista innanzitutto un'analisi dei numeri di porta validi, per sorgente e destinazione, classificandole secondo gli intervalli comunemente utilizzati, al fine di applicare successivamente eventuali regole sulle porte autorizzate.
Per TCP si verificano porte, disponibilità dei flag, lunghezza dell'intestazione e intervallo di numeri di sequenza e di riscontro. Si registrano anche i controlli mirati sulle flag di controllo fatti per individuare assenze o combinazioni particolari, senza ovviamente attribuirne un significato malevolo.
Per UDP si fanno controlli su disponibilità dei campi e di coerenza tra lunghezza dichiarata nell'intestazione di trasporto e le informazioni ricavate a livello IP. La verifica si basa sulla lunghezza minima di UDP di 8 byte, verificando la dimensione complessiva, che comprende intestazione e payload: se non soddisfatta denota un'incoerenza strutturale. Si valutano anche condizioni di frammentazione per individuare situazioni di incompletezza.
Per ICMP e ICMPv6 si fanno controlli sulla disponibilità dei campi di tipo e codice, verificabili anche in casi di cattura insufficiente, e in seguito sulla lunghezza del messaggio. Si distinguono così richieste o risposte Echo oppure messaggi che segnalano condizioni di errore.
Un altro aspetto che viene considerato è quello della classificazione del payload: nel momento in cui grazie alle informazioni disponibili si riesce a stabilire che il pacchetto è effettivamente vuoto si assegna l'etichetta adibita EMPTY PAYLOAD. Casi in cui il payload risulta assente a causa di catture parziali vengono rilevati per mantenere classificazioni coerenti con le informazioni osservabili.

4.2 Analisi di flusso

L'analisi basata sui flussi di rete ha lo scopo di andare oltre la valutazione dei singoli pacchetti attraverso le rappresentazioni aggregate, orientandosi alle verifiche sulle relazioni di pacchetti appartenenti alla stessa comunicazione, ricavandone misure statistiche e temporali. Affiancarla all'analisi di pacchetto risulta molto utile in situazioni in cui una serie di pacchetti singolarmente validi possono appartenere ad un flusso con comportamenti anomali.
I pacchetti vengono raggruppati in base a quella che viene definita come FlowKey, una chiave univoca e indipendente dalla direzione del traffico, che identifica il flusso, costruita sulla base della quintupla con versione IP, numero di protocollo IP, protocollo di trasporto identificato e i due endpoint coinvolti. Ogni endpoint è caratterizzato da un indirizzo IP e, per TCP e UDP, una

porta.
La normalizzazione degli indirizzi e la disposizione degli endpoint in ordine canonico vengono fatte per rappresentare il traffico inviato e quello ricevuto mediante un unico flusso con la stessa chiave, pur mantenendo contatori di pacchetti e byte separati per le due direzioni.
Vengono effettuate verifiche sulla coerenza degli endpoint, del protocollo, delle dimensioni dei pacchetti. Per ciascun flusso vengono progressivamente raccolte informazioni relative al volume di traffico, al numero e alle dimensioni dei pacchetti, alla durata della comunicazione e agli intervalli temporali tra pacchetti consecutivi.
Per traffico di tipo TCP vengono conteggiate le occorrenze di SYN, FIN e RST, utili per individuare aspetti dell'andamento della comunicazione. Viene mantenuto in modo semplificato lo stato per verificare un avvenuto handshake. A partire dal primo SYN si memorizza la direzione sorgente e l'handshake risulta osservato nel momento in cui si verifica la normale sequenza di SYN, SYN + ACK provenienti dalla direzione opposta e un ACK da parte del mittente. Un ACK ricevuto prima della combinazione SYN e ACK o inviato dal destinatario non completa la sequenza. Se la cattura inizia a connessione già avviata, lo stato rimane indeterminato. Restano fuori dall'implementazione la verifica dei sequence e acknowledge numbers e il tracciamento della chiusura della connessione, lo stato per ora rimane quindi indeterminato.
Un aspetto interessante considerato è quello riguardante l'ordine temporale: si verifica infatti la presenza di un timestamp utilizzabile, per ordinare innanzitutto tutti i pacchetti. Si ricavano quindi la durata del flusso come la differenza tra il primo e l'ultimo pacchetto e gli intervalli tra i pacchetti con le differenze tra timestamp consecutivi. Il timestamp costituisce una misura utile anche alla finalizzazione dei flussi, che può avvenire per inattività, oltre un timeout configurabile, o al termine della cattura. A timeout abilitato, si effettuano controlli periodici sui flussi attivi e un controllo sulla chiave del pacchetto corrente. Nel momento in cui un flusso viene finalizzato per inattività ma nel traffico catturato arriva un eventuale pacchetto con la stessa chiave, si ricrea una nuova istanza di quel flusso. Un pacchetto con timestamp assente o invalido viene escluso dall'aggregazione, pur rimanendo associato al proprio risultato di analisi singola, poiché non risulta collocabile nella sequenza temporale.
Si effettua anche dei controlli di consistenza dell'aggregazione, verificando che la somma dei contatori di pacchetti nelle due direzioni coincida con il numero totale di pacchetti del flusso e che la somma dei contatori di byte coincida con il totale dei byte.
In fase di finalizzazione si produce un oggetto FlowAnalysis che raccoglie tutte le statistiche, il motivo della chiusura e gli esiti dei controlli di coerenza.

5 Policy Engine

Il Policy Engine rappresenta il nucleo decisionale del firewall e la definizione delle regole di filtraggio ha costituito una delle principali scelte progettuali.
L'engine riceve gli esiti delle analisi e li confronta con un pool di limiti definiti nella configurazione. In base alle caratteristiche valutate, verifica una serie di controlli e stabilisce l'esito positivo o negativo sull'applicazione della policy.

5.1 Ordine di valutazione e politica di default

La configurazione è stata sviluppata sulla base della politica di default-deny, in modo tale che il sistema si comportasse assumendo la potenziale pericolosità di ogni pacchetto, respingendolo fino alla conferma di una specifica regola. Pur ribadendo la natura locale dell'elaborazione e il fatto di operare su traffico già acquisito, anche in questo caso le motivazioni di sicurezza hanno portato a prediligere questa politica, intercettando e bloccando traffico non autorizzato anche in casi sconosciuti. Ogni pacchetto viene quindi autorizzato solo se corrisponde ad un servizio esplicitamente ammesso oppure ad una risposta associata ad una richiesta già consentita.
La policy stabilisce anche quindi delle autorizzazioni di servizio e insieme dei limiti quantitativi e delle condizioni di blocco.
I limiti definiti stabiliscono una serie di valori massimi per le misure aggregate dei flussi, come durata, volume complessivo in byte e numero medio di pacchetti al secondo, oltre a delle soglie per il conteggio degli eventi nelle finestre temporali.
Le autorizzazioni di servizio sono la componente principale della policy dato che definiscono quali comunicazioni possono essere ammesse. Una regola specifica il protocollo, gli indirizzi, le reti

di origine e destinazioni, eventuali porte TCP/UDP oppure tipo e codice ICMP. L'applicazione dell'autorizzazione viene fatta dall'engine andando a verificare che i campi richiesti, validati nell'analisi di pacchetto, corrispondano ai criteri stabiliti.
Le autorizzazioni esplicite sono affiancate da alcune regole di blocco, che applicano dei vincoli in base a tre aspetti fondamentali: le caratteristiche del singolo pacchetto, il comportamento complessivo del flusso e la concentrazione di eventi negli intervalli temporali. Le condizioni di blocco specificano quando il confronto con i valori limiti o queste caratteristiche deve produrre un esito DROP. ed è l'ordine di valutazione che stabilisce quale regola viene riportata nel momento in cui allo stesso pacchetto sono applicate più condizioni di blocco. Si dà precendenza a controlli su timestamp, seguiti da quelli su tunnel non supportati, indirizzi IP non validati e verifiche strutturali. In ordine, i controlli su frammentazione, porte, flag TCP e dimensioni IP, limiti di flusso e su finestra temporale sono a seguire. Il primo blocco 'soddisfatto' è quello che inizializza la motivazione del DROP, rendendo disponibili tutti gli altri riscontri all'interno del risultato. La mancanza di autorizzazione produce un DROP, in linea con la politica di default deny. In assenza di condizioni di blocco soddisfatte, l'autorizzazione applicabile produce ALLOW.

5.2 Regole su pacchetto, flusso e finestra temporale

Le regole sono raggruppate in base alle informazioni valutate.

5.2.1 Regole per singolo pacchetto

Il primo livello di regole considera i campi osservati e gli esiti delle verifiche strutturali.

Timestamp utilizzabile Il timestamp costituisce un requisito obbligatorio in quanto riferimento comune per ordinamento degli eventi e applicazioni di regole temporali. Nel momento in cui il timestamp è assente, non numerico, booleano, negativo o non finito il pacchetto risulta inadatto all'elaborazione e riceve un esito DROP con motivazione INVALID TIMESTAMP.

Vincoli su indirizzi IP Si stabiliscono una serie di sorgenti o destinazioni non autorizzate con cui confrontare l'indirizzo del pacchetto: pacchetti provenienti da o diretti verso determinati host o intere reti specificati ricevono DROP.
Indirizzi multicast o broadcast utilizzati come sorgente sono bloccati. Sono ammessi gli indirizzi di loopback solo quando la cattura è dichiarata sull'interfaccia apposita ma bloccati in contesti di rete previsti dalla configurazione. In questo modo si applica un filtro contro pacchetti malformati, configurazioni errate o eventuali tentativi di IP spoofing. In casi in cui la cattura viene dichiarata come traffico WAN in ingresso o in uscita è possibile specificare i prefissi delle sorgenti ammessi. Il confronto avviene tra la sorgente del pacchetto e i prefissi definiti e blocca quelle che non corrispondono. In questo modo si applica un vincolo sulla provenienza attesa del traffico.

Vincoli su porte Si stabiliscono alcune porte sorgente o destinazione vietate, definite separatamente per TCP e UDP. Si applicano restrizioni alle porte 23, 135, 139 445 e la UDP 3389 rispettivamente associate a Telnet, RPC, NetBIOS, SMB e RDP, servizi di accesso remoto, legacy e condivisione in rete che possono costituire delle buone superfici di attacco per accessi non autorizzati, sfruttamento di vulnerabilità o movimenti laterali. Anche in caso di autorizzazioni più ampie, la scelta permette di limitare queste comunicazioni.

Controlli e requisiti Si applicano una serie di esiti anche in base a dei controlli strutturali bloccanti o dei requisiti espliciti. Nel momento in cui un determinato controllo richiesto è stato valutato come un'incoerenza con la flag INVALID nell'analisi di pacchetto, quindi risulta come una violazione accertata, si applica un blocco esplicito. Nel caso dei requisiti, il pacchetto riceve DROP in casi di verifica incompleta o indisponibile. Inserire questo tipo di regola è stato finalizzato a prevenire errori ma anche situazioni di pacchetti intenzionalmente malformati.

Tunnel IP non supportati Il prototipo di firewall valuta il traffico senza gestire la presenza di tecniche di incapsulamento: viene considerato il livello IP esterno e vengono bloccati gli incapsulamenti IPv4-in-IP, IPv6-in-IP e GRE riconosciuti con l'etichetta UNSUPPORTED TUNNEL.

Di conseguenza, non provvedendo di informazioni sufficienti, è stata introdotta questa regola per distinguere la casistica dalla semplice assenza di permesso.

Frammentazione IP La ricostruzione dei frammenti è un altro elemento non gestito dal prototipo. Il motore verifica se il pacchetto appartiene ad un datagramma frammentato e in caso affermativo li blocca. Il DROP viene prodotto anche in casi in cui le informazioni riguardanti la frammentazione sono mancanti o incoerenti. I pacchetti IPv6 contenenti il Next Header per la frammentazione e che trasportano il datagramma completo sono considerati ammessi.

Combinazioni flag TCP Un aspetto interessante è stato quello di applicare dei blocchi a combinazioni specifiche di flag TCP nel singolo segmento. In questo caso ricevono decisione DROP segmenti che presentano nessuna flag impostata oppure flag SYN e FIN o SYN e RST, combinati in casi di scansioni di host, fingerptinting o tentativi di evasione dei filtri. Quest'ultima viene applicata solamente quando il campo è stato letto correttamente e tutti i flag sono disattivati. La mancanza o la non interpretabilità dei flag non permettono la verifica.

Limite su dimensione IP Si definisce un limite opzionale e configurabile come massimo sulla dimensione del pacchetto a livello IP, analizzando la lunghezza totale dichiarata per IPv4 e i byte di intestazione con la lunghezza del payload per IPv6. Dimensioni eccessive rappresentano situazioni anomale ma non necessariamente sospette, l'inserimento della soglia è stato soprattutto a fine operativo rispetto alla politica adottata.

5.2.2 Regole per flusso

Le regole riguardanti il flusso utilizzano le statistiche dell'istanza bidirezionale finalizzata per produrre informazioni generali sui pacchetti che vi appartengono. Le soglie in pacchetti e byte al secondo sono applicate quando il flusso raggiunge una durata minima e un numero minimo di pacchetti, entrambi valori configurabili.
L'engine riporta anche delle segnalazioni riguardanti l'handshake TCP: un'etichetta di handshake incompleto indica che è stato osservato un tentativo di apertura ma non completato mentre un'anomalia segnala una combinazione SYN + ACK nella direzione del mittente.

Packet Rate Si confronta il numero medio di pacchetti al secondo con un valore massimo configurato, per individuare flussi con una frequenza di pacchetti elevati al fine di prevenire attività di flooding per sovraccaricare il servizio o comunicazioni automatizzate e ripetitive, con numerosi pacchetti di piccole dimensioni.

Byte Rate Si fa un confronto tra il numero medio di byte trasferiti in un unità di tempo con il relativo massimo. Il suo inserimento è finalizzato a limitare l'intensità del trasferimento anche in casi in cui il numero di pacchetti è contenuto, che potrebbero indicare utilizzi anomali di banda o trasferimenti significativi di dati in intervalli ridotti.

Flow Duration Corrisponde all'intervallo tra il primo e l'ultimo pacchetto, per verificare se la durata ammessa viene superata. Comunicazioni per intervalli di tempo prolungati vengono spesso associate a sessioni persistenti o automatizzate mantenute attive per periodi anomali o canali di comunicazione continuativi e risulta fondamentale porvi un limite.

Byte Count Si confronta la quantità totale di byte complessivi scambiati nelle due direzioni con il volume massimo ammesso, per prevenire utilizzi eccessivi di risorse, trasferimenti di grandi quantità di dati. Talvolta un volume elevato rischia di essere un indice per possibili attività di esfiltrazione di dati.

5.2.3 Regole per finestre temporali

Il terzo livello di regole valuta gli eventi osservati fino ad un certo pacchetto, a prescindere dall'appartenenza ai flussi.
La valutazione delle finestre temporali ha lo scopo di individuare concentrazioni di eventi in un determinato intervallo, costruito sulla base di una durata configurata. I pacchetti su cui si opera

sono in ordine cronologico. Si analizzano situazioni di flood di piccole dimensioni in cui il blocco viene imposto quando il conteggio supera una certa soglia stabilita. Nelle scansioni la funzione della soglia è quella di indicare il numero di porte o host distinti a partire dal quale si attiva la regola. Si utilizzano anche contatori di occorrenze per ciascuna porta o destinazione.
Situazioni malevole associabili ai flood sono quelle di possibili attacchi Dos mentre per le scansioni si rischiano fingerprinting, host discovery o network scanning.

Packet Flood Si contano i pacchetti per sorgente, destinazione e protocollo. La sua introduzione come vincolo ha l'obiettivo di individuare elevate concentrazioni di pacchetti tra stessi host in brevi intervalli.

SYN flood Si fa un conteggio dei segmenti TCP con SYN senza ACK e payload vuoto, raggruppandoli per sorgente, destinazione e porta di servizio, al fine di rilevare continui tentativi di apertura in direzione dello stesso servizio.

Echo Flood Le richieste Echo con codice 0 verificato vengono contate, raggruppando gli eventi per sorgente e destinazione. Richieste di questo tipo concentrate nel tempo potrebbero rappresentare la volontà di sovraccarico di un host o il consumo delle risorse di rete.

Scansione di porte Il conteggio viene applicato sul numero di porte destinazione distinte contattate dalla stessa sorgente sullo stesso host. In base al protocollo di trasporto si configurano finestre e soglie differenti; nel caso di TCP si considerano i SYN senza ACK e i segmenti con FIN senza SYN, RST o ACK mentre per UDP si considerano i datagrammi diretti alle diverse porte.

Scansione di host Il contatto con più indirizzi IP differenti può indicare attività di esplorazione della rete. Si provvede il conteggio degli indirizzi IP con destinazione distinta contattati da una stessa sorgente, anche in questo caso separando i conteggi per protocollo.

Sequenze di piccoli pacchetti Per sorgente, destinazione e protocollo si contano i pacchetti la cui lunghezza IP rientra nel massimo configurato. La concentrazione di numerosi pacchetti di dimensioni ridotte nella finestra può indicare tentativi di flooding senza necessariamente generare volumi elevati di byte.

5.3 Esito della valutazione

Per ciascun pacchetto, l'engine restituisce una decisione che esprime il rispetto o la violazione della policy adottata, tramite le etichette ALLOW e DROP. Insieme alla regola, vengono fornite una motivazione, dei riscontri secondari e anche gli identificativi delle regole pertinenti che però non è stato possibile valutare. In questo modo si riesce a ricostruire lo scenario del verdetto nel momento in cui numerosi controlli vengono applicati ed eventualmente parte dei dati non risulta disponibile.

6 Testing e risultati

6.1 Suite di test

Il corretto funzionamento del firewall e il suo comportamento sono stati verificati tramite una suite di 17 brevi file di cattura costruita ad hoc per il testing. Ogni file ha attraversato l'intera pipeline, dalla lettura dei pacchetti alla produzione delle decisioni. I risultati sono stati elaborati all'interno di report in JSON, contenenti gli ALLOW/DROP, le regole e ulteriori riscontri associati, per ogni pacchetto.
Per questa fase sono stati preparati una serie di profili di configurazione da introdurre nell'engine, diversificati per servizi autorizzati, restrizioni su indirizzi e porte, limiti sulle soglie, Cambiando semplicemente profilo si applicano stili diversi di politica alla stessa cattura. Per i test riportati è stato provveduto un apposito profilo dimostrativo con delle condizioni sperimentali e contenute: le finestre temporali sono state fissate ad una durata di 2 secondi; il massimo per il packet flood è di 30 eventi, per SYN flood, Echo flood e piccoli pacchetti sono stati impostati fino a 20 eventi; le scansioni si attivano a partire da 10 porte o host distinti; il limite sul volume complessivo di un

flusso è fissato a 60.000 byte.
Gli scenari introdotti sono stati elaborati con i seguenti obiettivi.

- **Test 1 - Comunicazione TCP** Verifica l'autorizzazione di una sequenza di handshake e scambio dati sulla porta 80 e del relativo traffico di risposta. Tutti i pacchetti ricevono ALLOW.
- **Test 2 - Richiesta e risposta UDP** Verifica che una richiesta con destinatario la porta 5353 autorizza la corrispondente risposta con indirizzi e porte invertiti nello stesso flusso. Entrambi i pacchetti ottengono esito ALLOW.
- **Test 3 - Servizio non autorizzato** Si invia un SYN verso la porta 23 per verificare l'applicazione della policy di default-deny. Il pacchetto riceve un DROP dato che è assente una regola di servizio applicabile.
- **Test 4 - SYN e FIN combinati** Si testa il blocco della combinazione delle due flag nello stesso segmento. Come previsto, il pacchetto riceve DROP.
- **Test 5 - SYN e RST combinati** Anche in questo caso si verifica la combinazione e il pacchetto riceve DROP.
- **Test 6 - Segmento privo di flag** Si verifica il riconoscimento di un segmento TCP con tutti i flag disattivati. Il pacchetto riceve DROP con motivazione di assenza di flag.
- **Test 7 - SYN seguito da RST** Verifica che il motore distingua una combinazione di flag nello stesso segmento da una richiesta seguita da un reset del destinatario. I pacchetti ricevono ALLOW
- **Test 8 - Scansione di porte TCP** Si inviano una serie SYN verso porte di destinazione diverse per verificare la correlazione tra flussi distinti. I primi 9 pacchetti ricevono ALLOW mentre tutti i successivi, a partire dalla decima porta distinta, ottengono DROP.
- **Test 9 - Scansione di porte UDP** Il controllo viene ripetuto con datagrammi UDP un numero di porte distinte. Anche in questo caso i primi 9 ricevono ALLOW e i restanti DROP.
- **Test 10 - Scansione di host** Si simulano un tot di tentativi verso host diversi, verificandone il conteggio, mantenendo stessa porta di destinazione. I primi 9 pacchetti sono consentiti e i restanti.
- **Test 11 - SYN flood** Si invia un numero significativo abbastanza grande di SYN verso lo stesso servizio con porte sorgente differenti. Coerentemente con le soglie definite, solo i primi 20 ricevono ALLOW.
- **Test 12 - Echo flood** Un tot di richieste ICMP Echo viene utilizzato per verificare la soglia sulle richieste ripetute. Anche in questo caso solo i primi 20 sono ammessi.
- **Test 13 - Piccoli pacchetti UDP** Si inviano 40 datagrammi da 29 byte a livello IP per verificare il conteggio dei pacchetti entro il massimo dimensionale di 80 byte. Come previsto, i primi 20 ricevono ALLOW e i successivi 20 DROP.
- **Test 14 - Packet flood UDP** Con un tot di datagrammi con payload di 200 byte si verifica il limite generale sui pacchetti: ricevono tutti gudizio ALLOW.
- **Test 15 - ACK puri** Si effettua una verifica con 40 segmenti TCP contenenti unicamente il flag ACK e con payload assente. I primi 30 pacchetti ricevono ALLOW mentre i restanti ricebono DROP per flooding della finestra temporale.
- **Test 16 - Accessi alla stessa porta** Attraverso un tot di SYN con porte sorgente diverse verso lo stesso host e servizio, si verifica che la scansione applichi il conteggio su porte di destinazione distinte. Tutti i pacchetti verificati sono ammessi.
- **Test 17 - Volume complessivo UDP** Si inviano 60 datagrammi nello stesso flusso che producono 74.520 byte, superando il massimo stabilito di 60.000, e tutti ricevono DROP.

6.2 Risultati

Attraverso la suite si riesce ad ottenere una verifica funzionale dei comportamenti rappresentati nei campioni, soprattutto a livello di coerenza con le condizioni configurate.
I primi test verificano aspetti semplici come l'autorizzazione dei servizi, la gestione delle risposte e l'applicazione della politica di default deny. I test sui flag permettono di verificare che un servizio ammesso rimane comunque soggetto di vincoli di blocco; i test sulle scansioni confermano invece il corretto funzionamento distinguendo eventi appartenenti a flussi diversi, contando le destinazioni distinte.
Nei test 11, 12 e 13 si osserva come, coerentemente con i limiti configurati, dal pacchetto n. 21 interviene il controllo specifico dello scenario mentre dal pacchetto n. 31 si considera il packet flood come la motivazione principale del DROP. In ogni test i primi 10 pacchetti sono ammessi mentre i successivi 10 sono attribuibili a situazioni di flooding.
Mantenendo anche i riscontri secondari si ha modo di osservare che l'interazione tra le regole e le condizioni di blocco avviene in modo corretto ma ci si rende conto anche che i campioni in questo modo risultano meno adatti a verificare in modo isolato ogni rilevatore.
Nell'ultimo test, il blocco di tutti i pacchetti dipende dalla valutazione del flusso finalizzati. Nel momento in cui viene riscontrato il superamento del volume massimo, si applica il giudizio all'intera istanza del flusso. Il totale considera le lunghezze originali dei pacchetti e comprende quindi anche le intestazioni.

7 Conclusioni

Il prototipo di firewall realizzato ha un comportamento coerente e corretto rispetto a tutte le regole stabilite e i limiti presenti. È interessante considerare una serie di sviluppi futuri possibili in funzione di eventuali migliorie.
Il funzionamento locale e offline permette di ripetere l'analisi sulla stessa cattura e confrontare gli esiti delle regole. Questo aspetto sicuramente costituisce un limite nel reale funzionamento del sistema e si potrebbe considerare di implementare l'elaborazione in tempo reale e l'acquisizione di traffico da differenti punti remoti, con conseguenti modifiche del codice per rivedere tutte quelle regole che utilizzano i dati del flusso e aggiornare le statistiche coerentemente con il tempo di trasmissione dei pacchetti.
Un altro aspetto emerso sicuramente è quello della scalabilità, legato al consumo di memoria: il lettore decodifica un pacchetto alla volte ma il sistema ne conserva le rappresentazioni in memoria. Il consumo di memoria quindi cresce proporzionalmente alla dimensione della cattura e con lo stato mantenuto per flussi e finestre temporali.
Nonostante tecnologie come gestione dell'incapsulazione o della frammentazione non siano state implementate per semplificazione, una versione futura potrebbe prevedere lo sviluppo di questi aspetti. Fondamentale dovrà essere l'accuratezza nel rifinire le regole dell'engine per questa implementazione: il tunneling è facilmente usato anche per aggirare controlli di filtraggio proprio che non ispezionano traffico incapsulato e allo stesso modo la frammentazione è sfruttata per nascondere informazioni rilevanti in frammenti successivi o introdurre sovrapposizioni.
Uno sviluppo particolarmente rilevante sarebbe quello di introdurre un terzo livello di analisi, basato sui protocolli applicativi. L'integrazione tramite il toolkit di nDPI permetterebbe di classificare i flussi anche in base a questo aspetto, utilizzabile dal Policy Engine come un ulteriore criterio di definizione ancora più specifico per le regole di autorizzazione e i blocchi.